

PAF DB design

Below is a **clean relational database structure** for the entity set you selected:

```
User
Role
Resource
ResourceType
Booking
Ticket
TicketAssignment
Comment
Notification
Attachment
```

The schema is designed so it works well with **Spring Boot + JPA + PostgreSQL/MySQL**.

1. Role Table

Defines user roles and permissions.

Table:

roles

Column	Type	Description
role_id	BIGINT (PK)	unique role identifier
role_name	VARCHAR(50)	ADMIN, USER, TECHNICIAN
description	TEXT	role description
created_at	TIMESTAMP	creation time

Example rows:

role_id	role_name
1	ADMIN
2	USER
3	TECHNICIAN

2. User Table

Stores system users.

Table:

users

Column	Type	Description
user_id	BIGINT (PK)	
name	VARCHAR(120)	
email	VARCHAR(120) UNIQUE	
role_id	BIGINT (FK → roles.role_id)	
department	VARCHAR(120)	
phone	VARCHAR(30)	
oauth_provider	VARCHAR(50)	
oauth_id	VARCHAR(120)	
status	VARCHAR(20)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Relationships:

`users.role_id → roles.role_id`

3. ResourceType Table

Defines types of resources.

Table:

resource_types

Column	Type	Description
type_id	BIGINT (PK)	
type_name	VARCHAR(100)	
category	VARCHAR(100)	
description	TEXT	

Example:

type_id	type_name	category
1	LECTURE_HALL	FACILITY
2	LAB	FACILITY
3	PROJECTOR	EQUIPMENT
4	CAMERA	EQUIPMENT

4. Resource Table

Represents facilities or equipment.

Table:

resources

Column	Type	Description
resource_id	BIGINT (PK)	
name	VARCHAR(150)	
type_id	BIGINT (FK → resource_types.type_id)	
location	VARCHAR(200)	
capacity	INT	

Column	Type	Description
status	VARCHAR(30)	
description	TEXT	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Example:

resource_id	name	type_id
1	Lecture Hall A401	1
2	Networking Lab	2

Relationship:

```
resources.type_id → resource_types.type_id
```

5. Booking Table

Represents reservations for resources.

Table:

bookings

Column	Type	Description
booking_id	BIGINT (PK)	
resource_id	BIGINT (FK → resources.resource_id)	
user_id	BIGINT (FK → users.user_id)	
start_time	TIMESTAMP	
end_time	TIMESTAMP	
purpose	TEXT	
expected_attendees	INT	
status	VARCHAR(30)	

Column	Type	Description
approval_reason	TEXT	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Booking statuses:

PENDING
APPROVED
REJECTED
CANCELLED

Relationships:

bookings.resource_id → resources.resource_id
bookings.user_id → users.user_id

6. Ticket Table

Represents maintenance incidents.

Table:

tickets

Column	Type	Description
ticket_id	BIGINT (PK)	
resource_id	BIGINT (FK → resources.resource_id)	
reported_by	BIGINT (FK → users.user_id)	
category	VARCHAR(100)	
priority	VARCHAR(20)	
description	TEXT	
status	VARCHAR(30)	

Column	Type	Description
preferred_contact	VARCHAR(120)	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Ticket statuses:

OPEN
 IN_PROGRESS
 RESOLVED
 CLOSED
 REJECTED

Relationships:

tickets.resource_id → resources.resource_id
 tickets.reported_by → users.user_id

7. TicketAssignment Table

Tracks technician assignments.

Table:

ticket_assignments

Column	Type	Description
assignment_id	BIGINT (PK)	
ticket_id	BIGINT (FK → tickets.ticket_id)	
technician_id	BIGINT (FK → users.user_id)	
assigned_by	BIGINT (FK → users.user_id)	
assigned_at	TIMESTAMP	

Relationships:

```
ticket_assignments.ticket_id → tickets.ticket_id
ticket_assignments.technician_id → users.user_id
ticket_assignments.assigned_by → users.user_id
```

8. Comment Table

Stores discussion comments on tickets.

Table:

comments

Column	Type	Description
comment_id	BIGINT (PK)	
ticket_id	BIGINT (FK → tickets.ticket_id)	
user_id	BIGINT (FK → users.user_id)	
content	TEXT	
created_at	TIMESTAMP	
updated_at	TIMESTAMP	

Relationships:

```
comments.ticket_id → tickets.ticket_id
comments.user_id → users.user_id
```

9. Notification Table

Stores user notifications.

Table:

notifications

Column	Type	Description
notification_id	BIGINT (PK)	
user_id	BIGINT (FK → users.user_id)	
type	VARCHAR(50)	
message	TEXT	
reference_id	BIGINT	
is_read	BOOLEAN	
created_at	TIMESTAMP	

Examples:

```
BOOKING_APPROVED
BOOKING_REJECTED
TICKET_STATUS_UPDATE
NEW_COMMENT
```

Relationship:

```
notifications.user_id → users.user_id
```

10. Attachment Table

Stores uploaded ticket images.

Table:

attachments

Column	Type	Description
attachment_id	BIGINT (PK)	
ticket_id	BIGINT (FK → tickets.ticket_id)	
file_name	VARCHAR(200)	
file_url	VARCHAR(300)	

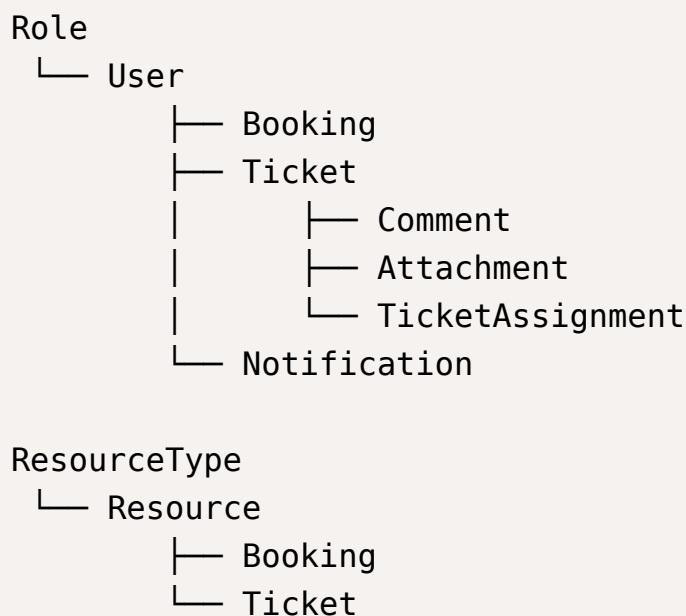
Column	Type	Description
file_type	VARCHAR(50)	
file_size	BIGINT	
uploaded_at	TIMESTAMP	

Relationship:

```
attachments.ticket_id → tickets.ticket_id
```

Overall Relationship Overview

Simplified ER structure:



Example SQL Schema (Simplified)

Example structure:

```
CREATE TABLE roles (
  role_id BIGINT PRIMARY KEY,
```

```
    role_name VARCHAR(50) UNIQUE NOT NULL,  
    description TEXT,  
    created_at TIMESTAMP  
);
```

```
CREATE TABLE users (  
    user_id BIGINT PRIMARY KEY,  
    name VARCHAR(120),  
    email VARCHAR(120) UNIQUE,  
    role_id BIGINT,  
    department VARCHAR(120),  
    phone VARCHAR(30),  
    oauth_provider VARCHAR(50),  
    oauth_id VARCHAR(120),  
    status VARCHAR(20),  
    created_at TIMESTAMP,  
    updated_at TIMESTAMP,  
    FOREIGN KEY (role_id) REFERENCES roles(role_id)  
);
```

You previously defined these entities:

```
User  
Role  
Resource  
ResourceType  
Booking  
Ticket  
TicketAssignment  
Comment
```

Notification
Attachment

Since the assignment **divides the system into four modules per member** , the entities should also be grouped by **which module owns them**. This is important because during the viva they may ask **which entities and endpoints you implemented**.

Below is the **clean modular ownership structure**.

Member 1 — Facilities & Assets Catalogue

Responsibility

Manage **all campus resources and asset metadata**.

Entities owned

Resource
ResourceType

Description

ResourceType

Defines categories of resources.

Examples:

- LECTURE_HALL
- LAB
- MEETING_ROOM
- PROJECTOR

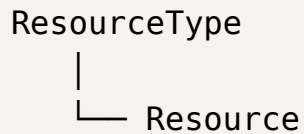
Resource

Represents a specific facility or equipment.

Examples:

- Lecture Hall A401
- Networking Lab B302
- Projector PRJ-01

Relationships



Member 2 — Booking Management

Responsibility

Handle **resource reservations and scheduling**.

Entities owned

Booking

Description

Booking

Represents a reservation for a resource.

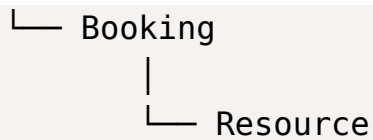
Key features:

- booking time slot
- approval workflow
- conflict prevention

Relationships

```
graph TD; U[User];
```

A diagram showing the text 'User' with a vertical line extending downwards from its center.



Booking workflow:

PENDING → APPROVED / REJECTED → CANCELLED

Member 3 — Maintenance & Incident Ticketing

Responsibility

Manage **incident reports and maintenance tracking**.

Entities owned

Ticket
TicketAssignment
Comment
Attachment

Description

Ticket

Represents a maintenance incident.

Example:

Broken projector in Lab 3

TicketAssignment

Tracks technician assignment.

Comment

Allows discussion on tickets.

Attachment

Stores images or files related to incidents.

Relationships

```
Ticket
├── TicketAssignment
├── Comment
└── Attachment
```

Member 4 — Authentication, Roles & Notifications

Responsibility

Manage **users, roles, access control, and system notifications.**

Entities owned

```
User
Role
Notification
```

Description

User

Represents people using the system.

Examples:

- Student
- Lecturer
- Technician

- Admin

Role

Defines permissions.

Examples:

```
ADMIN
USER
TECHNICIAN
```

Notification

System messages sent to users.

Examples:

- Booking approved
- Ticket updated
- New comment

Relationships

```
Role
|
└─ User
    └─ Notification
```

Final Ownership Map

```
Member 1
Facilities & Assets
├─ Resource
└─ ResourceType
```

Member 2

Booking Management

└─ Booking

Member 3

Maintenance & Ticketing

└─ Ticket

└─ TicketAssignment

└─ Comment

└─ Attachment

Member 4

Auth + Roles + Notifications

└─ User

└─ Role

└─ Notification

Entity Interaction Overview

Across modules, the system connects like this:

Role

|

User

└─ Booking

└─ Ticket

└─ Comment

└─ Attachment

└─ TicketAssignment

└─ Notification

ResourceType

|

Resource


```
graph LR; Root --- Booking; Root --- Ticket;
```

Booking

Ticket
